DS-UA 301
Advanced Topics in Data Science
Instructor: Parijat Dube, Chen Wang
Due: March 10th, 2026

Homework 2

General Instructions

This homework must be turned in on Gradescope by 11:59 pm on the due date. It must be your own work and your own work only—you must not copy anyone's work, or allow anyone to copy yours. This extends to writing code. You may consult with others, but when you write up, you must do so alone. Your homework submission must be written and submitted using Jupyter Notebook (.ipynb). **No handwritten solutions will be accepted**. You should submit:

1. One Jupyter Notebook(.ipynb) of your solutions for each of the problems in this assignment. (For example, you will need to submit **6** notebooks for this assignment.)

Please make sure your answers are clearly structured in the Jupyter Notebooks:

1. Label each question part clearly. Do not include written answers as code comments. The code used to obtain the answer for each question part should accompany the written answer.
2. All plots should include informative axis labels and legends. All codes should be accompanied by informative comments. All output of the code should be retained.
3. Math formulas can be typesetted in Markdown in the same way as LaTeX. A Markdown Guide is provided on Brightspace for reference.

For more homework-related policies, please refer to the syllabus.

Problem 1 - *Training a simple chatbot using a seq-to-seq model* 25 points

We will train a simple chatbot using movie scripts from the Cornell Movie Dialogs Corpus based on the **PyTorch Chatbot Tutorial** referenced below. This tutorial allows you to train a recurrent sequence-to-sequence model. You will learn the following concepts:

- Handle loading and pre-processing of the **Cornell Movie-Dialogs Corpus** dataset referenced below
- Implement a sequence-to-sequence model with **Luong attention mechanism(s)** referenced below
- Jointly train encoder and decoder models using mini-batches
- Implement greedy-search decoding module
- Interact with the trained chatbot

We will use the code in the tutorial as the starting code for this problem:

1. Make a copy of the notebook of the **PyTorch Chatbot Tutorial**, follow the instructions to train and evaluate the chatbot model in your local environment. **(3)**
2. Watch the video tutorial referenced below and learn how to use **Weights and Biases (W&B)** to run a hyperparameter sweep. You will instrument the notebook to use W&B to run some hyperparameter sweeps in the next steps.
3. Create a sweep configuration using the **W&B Random Search** strategy for the following hyperparameters: **(5)**
   - Learning rate: [0.0001, 0.00025, 0.0005, 0.001]

- Optimizer: [adam, sgd]
- Clip: [0, 25, 50, 100]
- teacher_forcing_ratio: [0, 0.5, 1.0]
- decoder_learning_ratio: [1.0, 3.0, 5.0, 10.0]

4. Run your hyperparameter sweeps using GPU-enabled environment and observe the results in the W&B console. **(5)**

5. Extract the values of the hyperparameters that give the best results (minimum loss of the trained model). Explain which hyperparameters affect the model convergence. Use the feature importance of W&B to help your analysis. Save the trained model that had the lowest loss. **(12)**

References:

- The Cornell Movie Dialogs Corpus dataset.
  Available at https://www.cs.cornell.edu/ cristian/Cornell_Movie-Dialogs_Corpus.html
- PyTorch Chatbot Tutorial.
  Available at https://pytorch.org/tutorials/beginner/chatbot_tutorial.html
- Luong et al. Effective Approaches to Attention-based Neural Machine Translation.
  Available at https://arxiv.org/abs/1508.04025
- Hyperparameter sweeps with Weights and Biases Framework tutorial.
  Video available at https://www.youtube.com/watch?v=9zrmUIlScdY
  Notebook available at https://colab.research.google.com/github/wandb/examples/blob/master/colabs/pytorch/Organizing_Hyperparameter_Sweeps_in_PyTorch_with_W%26B.ipynb
- Weights and Biases Website.
  Available at https://wandb.ai/site

Problem 2 - *Using BERT for Question Answering* 10 points

Please find this problem in the template provided on Brightspace.

Problem 3 - *Data Parallelism in Pytorch* 20 points

We are going to experiment with PyTorch's DataParallel Module, which is PyTorch's Synchronous SGD implementation across a number of GPUs on the same server. In particular, we will train ResNet-18 implementation from https://github.com/kuangliu/pytorch-cifar with num_workers=2, running up to 4 GPUs with the DataParallel (DP) Module. Use SGD optimizers with 0.1 as the learning rate, momentum 0.9, and weight decay 5e-4. For this question, you need to do experiment with multiple GPUs on the same server. You may need to execute this on the NYU Greene Cluster.

Create a PyTorch program with a DataLoader that loads the images and the related labels from the torchvision CIFAR10 dataset. Import the CIFAR10 dataset for the torchvision package, with the following sequence of transformations:

- Random cropping, with size 32x32 and padding 4
- Random horizontal flipping with a probability 0.5

Homework 2

- Normalize each image's RGB channel with mean(0.4914, 0.4822, 0.4465) and variance (0.2023, 0.1994, 0.2010)

The DataLoader for the training set uses a minibatch size of 128 and 3 IO processes (i.e., num_workers=2). The DataLoader for the testing set uses a minibatch size of 100 and 3 IO processes (i.e., num_workers =2). Create a main function that creates the DataLoaders for the training set and the neural network.

1. Measure how long it takes to complete 1 epoch of training using different batch sizes on a single GPU. Start from batch size 32, increase by 4-fold for each measurement (i.e., 32, 128, 512 ...) until single GPU memory cannot hold the batch size. For each run, run 2 epochs, the first epoch is used to warm up the CPU/GPU cache; and you should report the training time (*excluding* data I/O; but *including* data movement from CPU to GPU, gradients calculation and weights update) based on the 2nd epoch training. (5)

2. Measure running time with batch size per GPU you used in part 1 (i.e., 32, 128, ...) on 2 GPUs and 4 GPUs and calculate speedup for each setup. Again, for each setup, run 2 epochs, and only measure the 2nd epoch. When measuring speedup, one should include all the training components (e.g., data loading, cpu-gpu time, compute time). (5).
   Expected Answer: Table 1 records the training time and speedup for different batch sizes up to 4 GPUs. Comment on which type of scaling we are measuring: weak-scaling or strong-scaling? Comment on if the other type of scaling was used to speed up the number will be better or worse than what you are measuring.

3. Report for each batch size per GPU (i.e., 32, 128, 512 ...), how much time spent in computation (including CPU-GPU transferring and calculation), and how much time spent in communication in 2-GPU and 4-GPU case for one epoch. (hint You could use the training time reported in Question 1 to facilitate your calculation). (5)
   Expected Answer: First, describe how you get the compute and communication time in each setup. Second, list compute and communication time in Table 2.

4. Assume PyTorch DP implements the all-reduce algorithm as discussed in the class (reference below), and calculates communication bandwidth utilization for each multi-gpu/batch-size-per-gpu setup. (5)
   Expected Answer: First, list the formula to calculate how long it takes to finish an allreduce. Second, list the formula to calculate the bandwidth utilization. Third, list the calculated results in Table 3.

   References:

   - PyTorch Data Parallel, Available at website.
   - Bringing HPC Techniques to Deep Learning

	Batch-size 32 per GPU		Batch-size 128 per GPU		Batch-size 512 per GPU	
	Time(sec)	Speedup	Time (sec)	Speedup	Time (sec)	Speedup
1-GPU		1		1		1
2-GPU						
4-GPU						

Table 1: Speedup Measurement for different batch size.

	Batch-size 32 per GPU		Batch-size 128 per GPU		Batch-size 512 per GPU	
	Compute(sec)	Comm(sec)	Compute(sec)	Comm(sec)	Compute(sec)	Comm(sec)
2-GPU						
4-GPU						

Table 2: Compute and Communication time for different batch size.

	Batch-size-per-GPU 32	Batch-size-per-GPU 128	Batch-size-per-GPU 512
	Bandwidth Utilization(GB/s)	Bandwidth Utilization(GB/s)	Bandwidth Utilization(GB/s)
2-GPU			
4-GPU			

Table 3: Communication Bandwidth Utilization

Problem 4 - *Efficiency of Synchronous SGD Across Servers* 10 points

Consider a distributed deep learning experiment with data parallelism and synchronous SGD across multiple servers. Each server has 8 P100 GPUs and batch size per GPU is kept fixed at 64. The training dataset has 131072 images. *Scaling efficiency is defined as ratio of per iteration time when training using one 1 server to per iteration time when training using N servers.* The result from this experiment, showing how the per iteration time scales as the number of servers, are shown in Figure 1.

Number of servers	Time per iteration (secs)
1	0.3
2	0.32
4	0.33
8	0.35
16	0.36
32	0.37
64	0.39
128	0.41
256	0.43

Figure 1: Problem 5

Based on this data provide answer to the following. Note that for answers with decimal part you need to round to two places of decimal. **Show your work for parts 1-4**.

1. [**2 points**] Per epoch time (in secs) with 256 GPUs.
2. [**2 points**] Throughput (images/sec) with 256 GPUs.
3. [**2 points**] Per epoch time (in secs) with 1024 GPUs.

4. [**2 points**] Throughput (images/sec) with 1024 GPUs.
5. [**2 points**] Till 256 GPUs the scaling efficiency is greater than 85%. [**True/False**]

Problem 5 - *Update Time Analysis in Distributed Training* 10 points

Consider distributed training with 3 learners. Let the mini-batch processing times (in milliseconds) of six successive mini-batches at the three learners be given:

$$\begin{aligned} learner-1 &: 1.5, 0.9, 2.5, 1.2, 1.8, 0.9 \\ learner-2 &: 3, 2.5, 1.7, 3.0, 0.7, 0.8 \\ learner-3 &: 2.5, 1.5, 0.7, 0.9, 2.0, 2.2 \end{aligned}$$

Calculate the time to have three updates of the model parameters at the parameter server under following two algorithms:

1. [**5 points**] Sync (fully synchronous)
2. [**5 points**] Async

Problem 6 - *Prompt Engineering for Large Language Models* 25 points

In this problem, you will explore various prompt engineering techniques for Large Language Models (LLMs) using OpenAI's GPT models. You will implement and compare different prompting strategies to understand their effectiveness in different scenarios.

Setup your environment by installing the required libraries and configuring your OpenAI API key as demonstrated in the provided notebook template.

1. [**5 points**] Implement and compare Zero-Shot vs Few-Shot prompting for sentiment analysis:
   - Create a zero-shot prompt to classify the sentiment of the following texts as positive, negative, or neutral:
     - "The movie was absolutely fantastic and exceeded my expectations!"
     - "I'm not sure how I feel about this product."
     - "This service was terrible and completely disappointing."
   - Design a few-shot prompt with 2-3 examples for the same task and test it on the same texts.
   - Compare the results and explain which approach performs better and why.
2. [**5 points**] Implement Chain-of-Thought (CoT) prompting for mathematical reasoning:
   - Create a prompt that uses CoT to solve this problem: "A bakery sells cupcakes for $3 each and cookies for $2 each. If Sarah buys 4 cupcakes and 6 cookies, and pays with a $50 bill, how much change will she receive?"
   - Compare the CoT approach with a direct prompt (without step-by-step reasoning).
   - Analyze which approach provides more reliable and interpretable results.
3. [**5 points**] Experiment with Self-Consistency prompting:

- Design a prompt that generates multiple reasoning paths for this logic problem: "In a race with 5 participants (Alice, Bob, Carol, David, and Eve), Alice finished before Bob but after Carol. David finished last, and Eve finished before Alice but after Bob. What is the final ranking?"
- Generate at least 3 different reasoning paths using different temperature settings (0.3, 0.7, 1.0).
- Determine the most consistent answer across the different generations.

4. [**5 points**] Implement a simplified Tree of Thoughts (ToT) approach:
   - Create a prompt that systematically explores different approaches to solve this planning problem: "You need to pack for a 3-day camping trip. You have a backpack that can hold 5 items maximum. Available items: tent, sleeping bag, food (3 days worth), water bottles, flashlight, first aid kit, map, extra clothes, cooking equipment. Determine the optimal packing strategy."
   - Generate multiple "thought branches" by considering different priorities (safety-first vs. comfort-first vs. minimal-weight).
   - Evaluate and compare the different approaches to arrive at the best solution.
5. [**5 points**] Design and implement a Retrieval Augmented Generation (RAG) simulation:
   - Create a mock "retrieval content" about renewable energy sources (solar, wind, hydro).
   - Design a prompt that uses this retrieved information to answer the question: "What are the advantages and disadvantages of different renewable energy sources, and which would be most suitable for a small island nation?"
   - Compare the RAG approach with a prompt that relies solely on the model's pre-trained knowledge.
   - Discuss the benefits of using external knowledge sources in prompt engineering.
6. [**5 points**] *(Optional Bonus)* Implement an Automatic Reasoning and Tool-use (ART) agent:
   - Set up a ReAct (Reasoning and Acting) agent using LangChain that can access external tools such as web search and mathematical computation.
   - Configure the agent with at least two tools: a search tool (like Google Serper) and a calculator tool.
   - Test the agent with a multi-step reasoning task that requires both information retrieval and computation, such as: "What is the population of the largest city in Japan, and what would be 15% of that population?"
   - Document the agent's reasoning process, showing how it breaks down the problem, uses tools, and arrives at the final answer.
   - Analyze the advantages and limitations of tool-augmented LLMs compared to standalone prompting approaches.

For each part, you must:

- Provide the exact prompts you used
- Include the model's responses
- Analyze the effectiveness of each technique
- Discuss parameter choices (temperature, top_p, max_tokens) and their impact on results
- Present your findings in a clear, structured manner with appropriate markdown formatting

Homework 2

Note: If you observe minimal or no differences between different prompting techniques in your results, this is acceptable and expected behavior with state-of-the-art reasoning models like GPT-4. Modern LLMs have become increasingly robust and can often produce high-quality responses across various prompting strategies. Focus your analysis on understanding why certain techniques might be more suitable for specific types of problems, even if the performance differences are subtle.

Submit your work as a Jupyter notebook that demonstrates all the implemented techniques with clear explanations and analysis of the results.

References:

- Wei, J., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. arXiv:2201.11903.
- Wang, X., et al. (2022). Self-consistency improves chain of thought reasoning in language models. arXiv:2203.11171.
- Yao, S., et al. (2023). Tree of thoughts: Deliberate problem solving with large language models. arXiv:2305.10601.
- Yao, S., et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629
- Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. arXiv:2005.11401.
- OpenAI API Documentation. https://platform.openai.com/docs/